

東京大学 ブロックチェーン
イノベーション寄付講座

ADVANCED CRYPTOGRAPHY PROGRAM • WEEK 3

zkSNARK part 1

有限体 • 多項式 • Commitment • Arithmetization

nunocloth

目次

導入

- zkSNARK の導入
- 導入例

道具立て

- 数の準備
- 多項式の準備
- 楕円曲線/離散対数
- ペアリング

組み立て

- Commitment
- Arithmetization
- 対話 ZK/FS

締め

- まとめ
- グループワーク
- Appendix

zkSNARK で実現したいこと

秘密を手元に置いたまま、「条件を満たすものを知っている」という主張だけを相手に納得させたい。

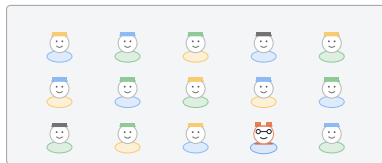
たとえばこんな場面

- パスワードを伏せたまま、正しいパスワードを知っていることを示す。
- 取引の中身を伏せたまま、ルールどおりの取引であることを示す (ブロックチェーンでの主な用途)。

登場人物は 2 人。示す側が証明者 (prover)、確かめる側が検証者 (verifier)。

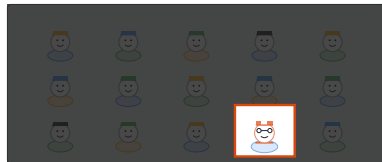
例 1: ウォーリーを探せ

「群衆の絵のどこかにウォーリーがいる」と示したい。ただし位置は秘密のまま。大きな紙で絵全体を覆い、ウォーリーだけが見える穴を開けて渡す。



prover だけが見ている絵
ウォーリーの位置を知っている

紙をかぶせる

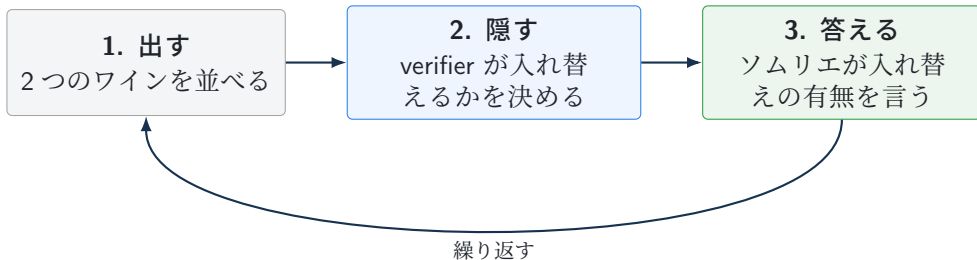


verifier へ渡す状態
穴からウォーリーだけが見える

verifier が受け取るのは「確かにいる」という事実だけ。絵のどこにいたかは prover の手元に残る。

例 2: ワインソムリエ

「2つのワインを飲み分けられる」と示したい。ただし見分け方のコツは秘密のまま。次の3手を1回とし、何度も繰り返す。



1回の正答なら偶然でも確率 $1/2$ 。10回続けて当てる偶然は $1/2^{10} < 0.1\%$ で、見分けていると考えるほうが自然になる。

例から用語を取り出す

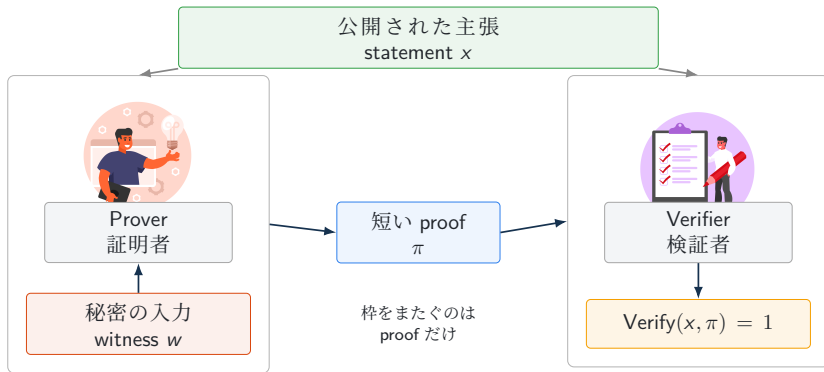
2つの例に共通する骨組みへ名前を付ける。

用語	ウォーリー	ソムリエ
statement(公開の主張)	群衆の中にウォーリーがいる	2つのワインを区別できる
witness(秘密)	ウォーリーの位置	味を見分けるコツ
proof(証拠)	穴から見える姿	当てっこへの正答の列

どちらの例でも、witness は prover の手元に残ったまま検証が終わる。

登場人物と流れ

以降、statement を x 、witness を w 、proof を π と書く。



witness w はこの枠の中に留まる

statement と witness を式で見る

公開値 $y = 15$ に対して、 $w^3 + w + 5 = y$ を満たす秘密の w を知っている、と主張したい。

statement: 公開される存在主張

$$\exists w : w^3 + w + 5 = 15$$

witness: 主張を支える秘密

$$w = 2$$

verifier へ渡るのは statement と proof だけで、解 $w = 2$ は prover の手元に残る。

主張を検査プログラム R で書く

主張の「条件」の部分は、公開入力 y と秘密の候補 w を受け取って合否を返す関係 R としてまとめられる。

```
def R(y, w):  
    # y: public value  
    # w: secret candidate  
    if w**3 + w + 5 == y:  
        return 1  
    return 0
```

$y = 15, w = 2$ なら $2^3 + 2 + 5 = 15$ なので $R(15, 2) = 1$ 。この記法にすると、statement は

$$\exists w \text{ such that } R(y, w) = 1$$

という形の存在主張にそろろう。zkSNARK が扱うのは、この形に書けた主張である。

zkSNARK とは

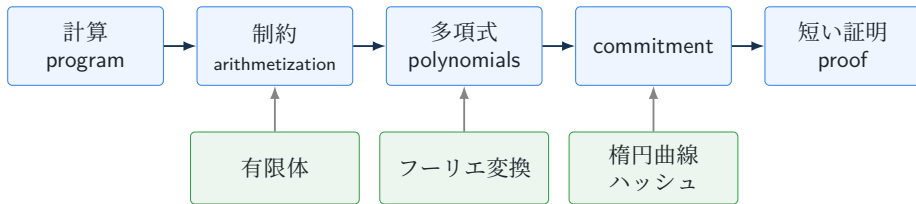
zkSNARK は、次の性質をすべて満たす証明システムの総称。頭文字がそのまま性質の一覧になっている。

- **zk** (Zero-Knowledge): proof から witness 由来の余計な情報が漏れない。ウォーリーの例の「位置は分からないまま」に当たる。
- **S** (Succinct): proof が短く、verifier は速く検証できる。
- **N** (Non-interactive): proof1 通で完結する。ソムリエの例で必要だった何往復もの当てっこを 1 通に置き換える。
- **ARK** (ARgument of Knowledge): 有効な proof を作れる prover は、対応する witness を知っている (knowledge soundness)。正しい witness を持つ prover の proof は受理される (completeness)。

Groth16、Plonk、Halo2 などが、この性質を満たすプロトコルの例。中身の道具立てが違っても、まとめて zkSNARK と呼ばれる。

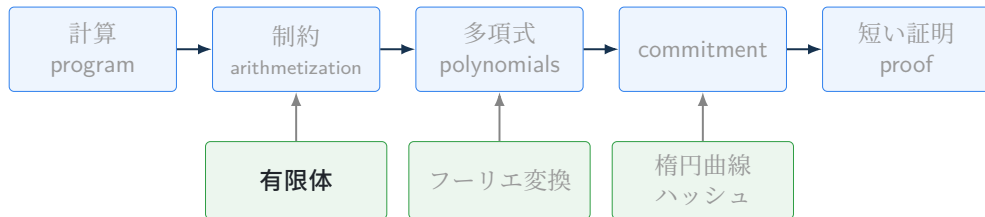
今日使う道具の全体像

zkSNARK は、計算の正しさの主張を段階的に変換して、短い proof へ落とし込む。



- **有限体**: すべての計算をこの中で行う。
- **多項式**: 多数の制約を少ない本数の等式へまとめる道具。係数と評価値の行き来をフーリエ変換が支える。
- **Commitment**: 多項式や値を短い値に固定し、あとからの書き換えを防ぐ。hash や楕円曲線を使う。

ここからの話: 数の準備



有限体: 余りの世界で計算する

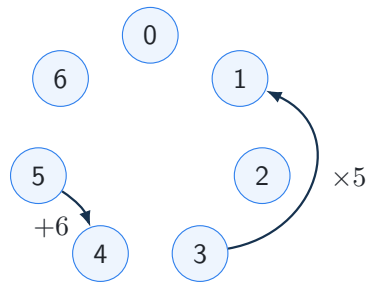
整数を7で割った余りだけを見る世界を考える。

$$\{0, 1, 2, 3, 4, 5, 6\}$$

計算結果が7以上になったら、7で割った余りへ戻す。「余りが等しい」ことを $\equiv$ で書く。

$$5 + 6 = 11 \equiv 4, \quad 3 \cdot 5 = 15 \equiv 1 \pmod{7}$$

足し算・引き算・掛け算は、この規則だけでいつでも計算できる。この集合を $\mathbb{Z}/7\mathbb{Z}$ と書く。



計算は、7個の点を巡る輪の上の移動になる。

法は素数にとる

法が素数 p のときに限り、0以外のすべての値に「掛けて1になる相手(逆元)」がそろい、割り算まで自由に行える。この世界 $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$ を有限体と呼ぶ。zkSNARKの計算は、大きな素数の $\mathbb{F}_p$ の中で行う。

有限体: 逆元と割り算

a の逆元 a^{-1} は、掛けると 1 になる値。割り算は、逆元を掛ける操作にあたる。

$$a \neq 0, \quad a \cdot a^{-1} = 1$$

a	1	2	3	4	5	6
a^{-1}	1	4	5	2	3	6

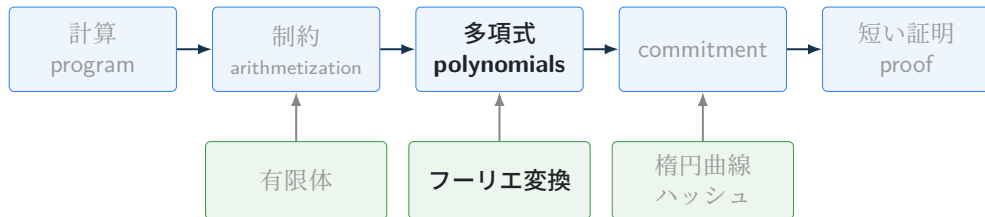
$\mathbb{F}_7$ の逆元表。例: $2 \cdot 4 = 8 \equiv 1 \pmod{7}$ 。

例: $\mathbb{F}_7$ で $3x = 5$ を解く

表から $3^{-1} = 5$ 。両辺に 5 を掛けて $x = 5 \cdot 5 = 25 \equiv 4 \pmod{7}$ 。検算: $3 \cdot 4 = 12 \equiv 5 \pmod{7}$ 。

逆元がそろうのは 0 以外の値。求め方は拡張 Euclid 互除法 (appendix)。

ここからの話: 多項式の準備



有限体から多項式へ

証明したい計算は、 $\mathbb{F}_p$ の元が並んだ長い列と、それらが満たすべきたくさんの等式として表される。この列と等式をまとめて扱う言語が多項式である。

有限体の元のまま見る

- 等式を 1 本ずつ確認する。
- 本数が増えるほど、検証の手間もそのまま増える。

多項式にまとめて見る

- $\mathbb{F}_p$ の元の列を 1 本が多項式で表す。
- 多数の等式が少数の多項式の等式になる。
- ランダムに選んだ 1 点の値を比べるだけで、全体をまとめて確かめられる。

右の道を通るために、まず多項式そのものの扱いを整理する。

多項式: 係数表現と評価表現

多項式

$$f(X) = 3X^2 + 2X + 5$$

は二つの見方を持つ。

係数表現

$(5, 2, 3)$

定数項から順に持つ。

評価表現

$(f(x_0), f(x_1), f(x_2))$

点での値の列として持つ。

重要な事実

次数 d の多項式は、異なる $d + 1$ 点での値が決まれば一意に決まる。係数表現の個数 $d + 1$ と、評価表現の個数 $d + 1$ は等しい。

多項式: 補間で点から式を復元する

評価表現から係数表現へ戻す操作を補間と呼ぶ。1 次式 $f(X) = aX + b$ なら、2 点で決まる。

$$f(1) = 3, \quad f(2) = 5$$

傾きは $a = \frac{5-3}{2-1} = 2$ 、切片は $b = 3 - 2 \cdot 1 = 1$ なので、

$$f(X) = 2X + 1$$

点数が増えても同じことができる。その一般の手順が Lagrange 補間 (appendix)。

フーリエ変換: 二つの表し方を行き来する

係数表現と評価表現を行き来する変換がフーリエ変換。評価点を規則的にそろえておくと、この行き来が扱いやすくなる。

位数 n の 1 の根

有限体 $\mathbb{F}_p$ の元 ω が

$$\omega^n = 1, \quad \omega^0, \omega^1, \dots, \omega^{n-1} \text{ がすべて異なる}$$

を満たすとき、 ω を位数 n の 1 の根と呼ぶ。

評価点は、この ω の冪にとる。

$$H = \{1, \omega, \omega^2, \dots, \omega^{n-1}\}$$

係数列 $a = (a_0, \dots, a_{n-1})$ から、 H 上の値の列 $y = (f(1), f(\omega), \dots, f(\omega^{n-1}))$ を作る操作が、有限体上のフーリエ変換。

フーリエ変換: 有限体上での定義

係数列 $a = (a_0, a_1, \dots, a_{n-1})$ は、次数 n 未満の多項式

$$f(X) = \sum_{i=0}^{n-1} a_i X^i$$

を表す。位数 n の 1 の根 ω に対して、有限体上のフーリエ変換と逆変換を次で定義する。

$$\text{フーリエ変換:} \quad y_j = f(\omega^j) = \sum_{i=0}^{n-1} a_i \omega^{ij}, \quad j = 0, \dots, n-1.$$

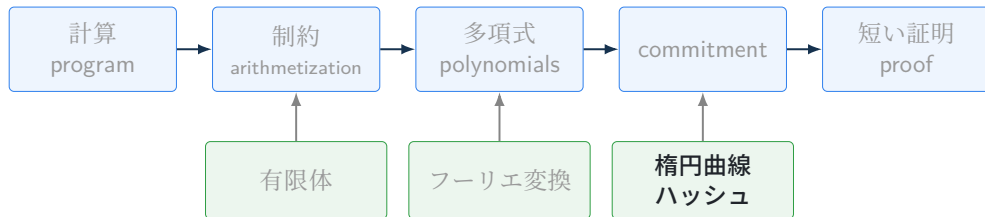
$$\text{逆変換:} \quad a_i = \frac{1}{n} \sum_{j=0}^{n-1} y_j \omega^{-ij}, \quad i = 0, \dots, n-1.$$

ここで $1/n$ は有限体の中での n の逆元。変換は係数列 a から評価値列 y を作り、逆変換は y から a を戻す。

計算量

定義どおりだと $O(n^2)$ 。 $O(n \log n)$ まで速くする手法が高速フーリエ変換 (FFT)(appendix)。

ここからの話: 楕円曲線/離散対数



群: 巡回群と位数

演算が集合の中で閉じていて、単位元と逆元があり、結合法則が成り立つ。この4つを満たす組を群と呼ぶ。 $\mathbb{F}_p^* = \mathbb{F}_p \setminus \{0\}$ は掛け算について群になる。

ある元 g を掛け続けて全体を巡れるとき、 g を生成元、この群を巡回群と呼ぶ。

例: $\mathbb{F}_7^*$ は $g = 3$ で巡れる

$$3^1 = 3, \quad 3^2 = 2, \quad 3^3 = 6, \quad 3^4 = 4, \quad 3^5 = 5, \quad 3^6 = 1 \pmod{7}$$

6 ステップで 0 以外の全要素が一度ずつ現れ、1 へ戻る。

- 群の位数: 群に含まれる要素の数。 $|\mathbb{F}_p^*| = p - 1$ 。
- 元 g の位数: $g^n = 1$ となる最小の正整数 n 。上の例では 3 の位数は 6。

多項式の準備で使った「位数 n の 1 の根」も、この位数の言葉で言い直せる。

楕円曲線: 点の足し算で群を作る

$\mathbb{F}_p^*$ は、掛け算について群になっていた。楕円曲線は、曲線上の点の「足し算」で群を作る、もう一つの方法である。

点の足し算

$$P + Q = R$$

点と点を足すと、また曲線上の点になる。

スカラー倍

$$xP = \underbrace{P + \dots + P}_{x \text{ 個}}$$

P を x 回足す操作。

この群が暗号の土台になるのは、 xP から x を逆算するのが難しいため。

楕円曲線: 式で見る

暗号でよく使う楕円曲線は、有限体 $\mathbb{F}_p$ 上の点集合として定義される。

$$y^2 = x^3 + ax + b$$

滑らかさの条件

$$4a^3 + 27b^2 \neq 0$$

この条件により、特異点のない曲線として扱える。

曲線上の点に、無限遠点 $\mathcal{O}$ を1つ加えたものを $E(k)$ と書く。

$$E(k) = \{(x, y) \in k^2 \mid y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$$

$\mathcal{O}$ は足し算の単位元として働く。

楕円曲線: 加法公式 (1/2) 3つ目の交点

$P = (x_1, y_1)$ 、 $Q = (x_2, y_2)$ とする。

定義

P, Q を通る直線が曲線と 3 つ目に交わる点を $R' = (x_3, y')$ とする。 R' の x 軸反転 $R = (x_3, -y')$ を

$$P + Q := R$$

と定める。

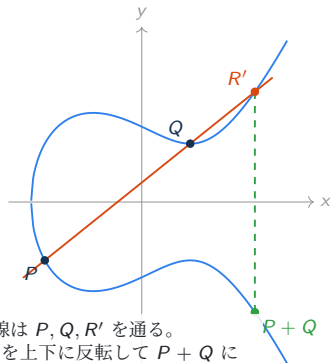
ここから x_3, y' を式で求める。 $P \neq Q$ かつ $x_1 \neq x_2$ の通常ケースでは、

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}$$

とおく。直線 $y = \lambda(x - x_1) + y_1$ を曲線 $y^2 = x^3 + ax + b$ に代入すると、

$$\{\lambda(x - x_1) + y_1\}^2 = x^3 + ax + b$$

という 3 次方程式になる。解は交点の x 座標 x_1, x_2, x_3 である。



直線は P, Q, R' を通る。
 R' を上下に反転して $P + Q$ にする。

図は実数上の直感。暗号では同じ式を有限体で $\text{mod } p$ 計算する。

楕円曲線: 加法公式 (2/2) 反転して和を作る

$P + Q = (x_3, y_3)$ の座標

$$x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

代入して得た3次方程式で x^2 の係数は $-\lambda^2$ になり、根と係数の関係から $x_1 + x_2 + x_3 = \lambda^2$ 。ここから1つ目の式が出る。3つ目の交点 $R' = (x_3, y')$ は直線上にあるので $y' = \lambda(x_3 - x_1) + y_1$ 。 $P + Q$ はこの x 軸反転なので、符号を返して2つ目の式になる。

$P = Q$ の点倍算

直線の代わりに接線を使う。

$$\lambda = \frac{3x_1^2 + a}{2y_1} \quad (2y_1 \neq 0)$$

$y_1 = 0$ なら接線は垂直になり、 $2P = \mathcal{O}$ と扱う。割り算は有限体の逆元で行う。

楕円曲線: 例 $\mathbb{F}_{11}$ 上で $2P$ を計算する

曲線 $y^2 = x^3 + x + 6$ ($\mathbb{F}_{11}$ 上、 $a = 1$) で、点 $P = (2, 7)$ の 2 倍を接線の式で計算する。

Step 1: 接線の傾き λ

$$\lambda = \frac{3x_1^2 + a}{2y_1} = \frac{3 \cdot 4 + 1}{14} = 13 \cdot 14^{-1} \equiv 2 \cdot 3^{-1} \equiv 2 \cdot 4 = 8 \pmod{11}$$

ここで $3^{-1} = 4$ は $3 \cdot 4 = 12 \equiv 1 \pmod{11}$ から。

Step 2: 座標を求める

$$x_3 = \lambda^2 - 2x_1 = 64 - 4 = 60 \equiv 5, \quad y_3 = \lambda(x_1 - x_3) - y_1 = 8 \cdot (2 - 5) - 7 = -31 \equiv 2$$

よって $2P = (5, 2)$ 。検算: $2^2 = 4$ 、 $5^3 + 5 + 6 = 136 \equiv 4 \pmod{11}$ で、たしかに曲線上の点。

楕円曲線: スカラー倍は倍々で速くなる

xP は P を x 回足す操作。定義どおりに足すと $x - 1$ 回の演算がかかる。実装では x の二進展開を使う。

例: $13P (13 = (1101)_2 = 8 + 4 + 1)$

$$P \xrightarrow{\text{doubling}} 2P \xrightarrow{\text{doubling}} 4P \xrightarrow{\text{doubling}} 8P, \quad 13P = 8P + 4P + P$$

doubling 3 回と足し算 2 回、合計 5 回の演算で届く。定義どおりなら 12 回かかる。

一般に演算回数は $\log_2 x$ に比例する。 x が 256bit 程度の巨大な数でも、数百回の演算で xP を計算できる。

楕円曲線: 離散対数問題

点 P を x 回足した xP は、倍々法で速く求まる。逆向きに戻せるかを問うのが次の問題。

P と $Q = xP$ から x を求める

これを楕円曲線上の離散対数問題と呼ぶ。

速い向き

$$x \mapsto Q = xP$$

難しい向き

$$P, Q = xP \mapsto x$$

hash と同じ見方になる。 $m \mapsto \text{Hash}(m)$ は速く、 $\text{Hash}(m)$ から m を戻すのは難しい。楕円曲線では、入力をスカラー x 、出力を点 xP としてこの構造を使う。

暗号では、基点 P が生成する大きな素数位数 q の部分群を使い、 $x \in \mathbb{Z}_q$ として扱う。

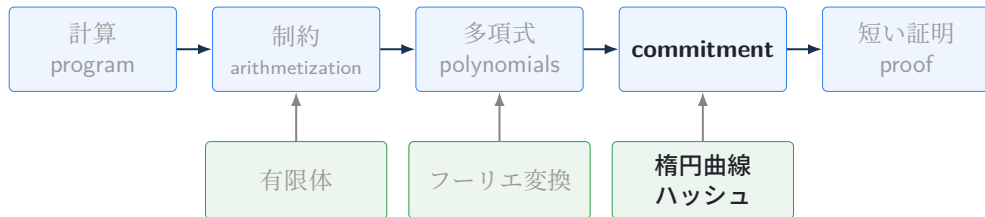
離散対数: 仲間の問題

離散対数問題の周りには、安全性を議論するときによく出てくる仲間がいる。Alice と Bob がそれぞれ aP 、 bP を公開して共通の abP を作る鍵共有を思い浮かべると覚えやすい。

問題	乗法記法	加法記法
離散対数	g, g^a から a を求める	P, aP から a を求める
計算 Diffie–Hellman	g, g^a, g^b から g^{ab} を求める	P, aP, bP から abP を求める
判定 Diffie–Hellman	g, g^a, g^b, T から $T = g^{ab}$ かランダムな g^c かを判定する	P, aP, bP, T から $T = abP$ かランダムな cP かを判定する

乗法記法の g^a と加法記法の aP は、同じ操作の書き方違い。どちらでも、やっていることは秘密スカラーを公開できる群要素へ移すことである。

ここからの話: ペアリング



ペアリング: なぜ要るのか

楕円曲線の点 aP は、スカラー a を隠したまま公開できるのだった。この後の commitment では、隠れたスカラーどうしの掛け算の関係を確かめたい場面が出てくる。

困りごと

aP と bP から足し算で作れるのは $aP + bP = (a + b)P$ まで。積にあたる abP は、その先にある。

ペアリングは、点のペアから「スカラーの積」を検査できる値を作る演算。分かるのは積が一致するかどうかまでで、離散対数の一方方向性はそのまま残る。

ペアリング: 定義

ペアリングは、二つの群の要素を別の群 G_T へ写す演算。 G_1, G_2, G_T の位数はどれも素数 q とする。

$$e : G_1 \times G_2 \rightarrow G_T$$

双線形性: スカラーを指数へ移せる

$$e(aP, Q) = e(P, Q)^a = e(P, aQ)$$

どちらの引数に付いたスカラーも G_T の指数として外へ出せる。両方まとめると $e(aP, bQ) = e(P, Q)^{ab}$ 。

- 非退化性: 生成元 P, Q に対して $e(P, Q) \neq 1$ 。
- 効率的計算可能性: P, Q から $e(P, Q)$ を効率よく計算できる。
- 実用では、 G_1, G_2, G_T を別々の群として扱う非対称ペアリングがよく使われる。

ペアリング: 積の関係を等式で確かめる

aP, bQ, cP を受け取った verifier が、「 $c \equiv ab \pmod{q}$ か」を確かめたいとする。双線形性で両辺を $e(P, Q)$ の指数に直すと

$$e(aP, bQ) = e(P, Q)^{ab}, \quad e(cP, Q) = e(P, Q)^c$$

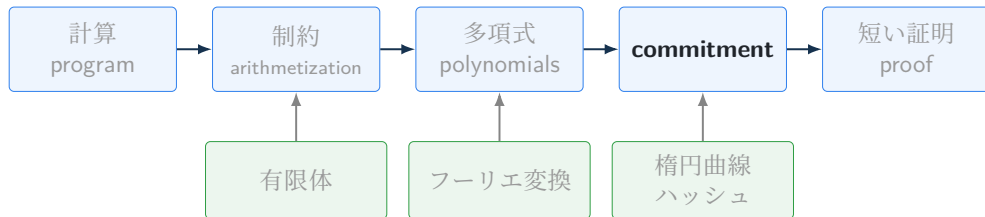
なので、

$$e(aP, bQ) = e(cP, Q) \iff ab \equiv c \pmod{q}$$

として検査できる。verifier が手に取るのは点だけで、スカラー a, b, c は隠れたまま。

次のセクションで見る KZG commitment の検証式 $e(C - yP, Q) = e(\pi, \tau Q - zQ)$ は、この「指数の等式をペアリングで確かめる」形そのものである。

ここからの話: Commitment



Commitment: zkSNARK での役割

証明システムでは、大きな対象が何度も出てくる。

- 制約の列
- 実行トレース
- 多項式
- wire 値や selector



開示する部分だけを
検証対象にする

役割

commitment は、大きな対象を短い値で代表させ、必要な点だけを open するために使う。
verifier が見るのは、その短い値と開かれた部分だけになる。

Commitment: scheme

値 m を短い c に固定して先に渡し、後で中身を見せて開ける。封をした封筒と同じ扱い。



binding

open できるのは、最初に入れた m だけ。

hiding

open するまで、verifier が持つのは c だけ。

基本形では全体を開く。KZG のように、1 点の値だけを補助情報つきで open する構成もある。

Hash commitment

message m を hash 値 1 個に固定する。

$$c = \text{Hash}(m)$$

prover は c を先に公開し、open では m を渡す。verifier は $\text{Hash}(m) = c$ を確認する。

binding

別の $m' \neq m$ で $\text{Hash}(m') = c$ となる組を見つけるのが難しい、という hash 関数の性質（第二原像困難性・衝突困難性）が支える。

hiding

m の候補が少ないと、手当たり次第 hash して c と見比べるだけで中身が分かる。十分長い乱数 r を混ぜると、候補は r の取りうる範囲まで一気に広がる。

$$c = \text{Hash}(r, m)$$

KZG: setup

KZG の setup では秘密のスカラー τ を 1 つ選び、次の群要素を公開する。

$$\text{SRS} = (P, \tau P, \tau^2 P, \dots, \tau^d P; \quad Q, \tau Q)$$

- $P \in G_1, Q \in G_2$: それぞれの生成元。位数は素数 q 。
- $\tau \in \mathbb{F}_q$: setup で選ぶ秘密。公開されるのは点だけで、 τ 自体は誰も知らない。
- d : commit できる多項式の次数の上限。

この形の SRS は MPC (ceremony) で作れる。参加者が順番に自分の乱数 τ_i を掛けて次へ渡すと、全体の秘密は $\tau = \tau_1 \tau_2 \cdots \tau_n$ になる。1 人でも自分の τ_i を捨てれば、 τ は誰にも分からない。

KZG: commitment

次数 d 以下の多項式を、 G_1 の点 1 個に固定する。

$$f(X) = a_0 + a_1X + \cdots + a_dX^d$$

commitment は、係数を SRS の点に掛けて足したものの。

$$C = a_0P + a_1(\tau P) + a_2(\tau^2P) + \cdots + a_d(\tau^dP) = f(\tau)P$$

- 使うのは SRS の点と係数 a_i だけ。 τ を知らないまま $f(\tau)P$ が手に入る。
- C は点 1 個。多項式の次数がいくつでもサイズは同じ。
- 係数を 1 つでも変えれば C も変わる。 C を出した時点で f は 1 つに決まる。

あとは、この C について「 $f(z) = y$ だ」と言えるようにすればよい。

KZG: opening

$f(z) = y$ を示したい。 $f(z) - y = 0$ なので $f(X) - y$ は $X = z$ を根に持ち、 $X - z$ で割り切れる。

$$q(X) = \frac{f(X) - y}{X - z} \quad \text{つまり} \quad f(X) - y = q(X)(X - z)$$

prover: 商多項式 q を commit した点を proof として出す。SRS の点の線形結合で作れる。

$$\pi = q(\tau)P$$

verifier: $X = \tau$ を入れた $f(\tau) - y = q(\tau)(\tau - z)$ が成り立つかを、ペアリングで確かめる。

$$e(C - yP, Q) = e(\pi, \tau Q - zQ)$$

正しく作った proof なら、両辺は次のようにつながる。

$$\begin{aligned} e(C - yP, Q) &= e((f(\tau) - y)P, Q) = e(q(\tau)(\tau - z)P, Q) \\ &= e(q(\tau)P, (\tau - z)Q) = e(\pi, \tau Q - zQ) \end{aligned}$$

使ったのは「スカラーをどちらの引数からも指数へ移せる」規則だけ。 C も π も点 1 個で、多項式の次数によらず定数サイズ。

KZG: 例 $f(X) = X^2 + 1$ で $f(2) = 5$ を開く

Step 1: commitment

$$C = f(\tau)P = \tau^2 P + P$$

多項式 $f(X) = X^2 + 1$ を固定する。SRS にある $\tau^2 P$ と P を足すだけで、 τ を知らずに計算できる。

Step 2: 商多項式と proof

$$q(X) = \frac{f(X) - 5}{X - 2} = \frac{X^2 - 4}{X - 2} = X + 2, \quad \pi = q(\tau)P = \tau P + 2P$$

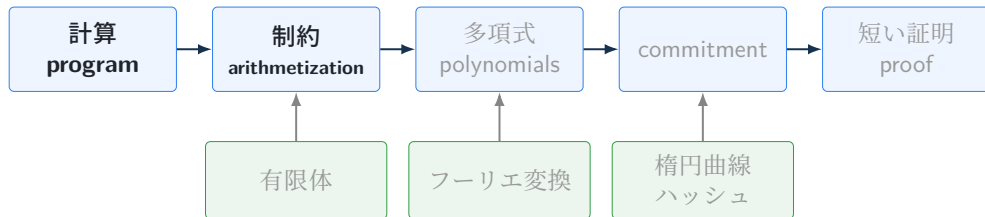
割り切れるのは、 $f(2) = 5$ が本当に成り立つときだけ。

Step 3: 検証

$$e(C - 5P, Q) = e(\pi, \tau Q - 2Q)$$

左辺の指数は $f(\tau) - 5 = \tau^2 - 4$ 、右辺の指数は $(\tau + 2)(\tau - 2) = \tau^2 - 4$ で一致する。

ここからの話: Arithmetization



Arithmetization: 目的

「計算が正しく実行された」という主張を、有限体上の制約として書き表す工程。

ここまでに用意した多項式や commitment が扱えるのは、有限体の等式として書かれた主張。証明したい計算をその形へ翻訳するのが、この節の仕事になる。

zk アプリケーション開発では、自分が書いたコードがどんな制約になるかが、そのまま証明の重さに響く。



書いたコードが、
そのまま制約の量になる

Arithmetization: 制約は「0になる式」

制約は、候補の値を代入したときに成り立つべき条件。verifier は計算をもう一度実行する代わりに、値どうしの整合性を等式で確認する。

計算として読む

$$a + b = c$$

c は $a + b$ の結果である。

制約として読む

$$a + b - c = 0$$

与えられた a, b, c が整合しているかを確認する。

確認しやすいように、成り立つべき関係は左辺が 0 になる形へそろえておく。

$$\underbrace{ab - c = 0}_{\text{掛け算}}$$

$$\underbrace{a + b - c = 0}_{\text{足し算}}$$

$$\underbrace{b(b - 1) = 0}_{b \in \{0,1\}}$$

たとえば AND は $ab - c = 0$ で書ける。このとき a, b が 0 か 1 であることも、 $b(b - 1) = 0$ の形で別の制約として入れておく。

Arithmetization: witness と中間値を制約へ分解する

次の計算を証明したい。

$$w^3 + w + 5 = y$$

verifier にも見えるのは公開値 $y = 15$ 。prover は秘密入力 w と中間値 t_1, t_2 を witness として持つ。

計算ステップへ分解する

$$t_1 \leftarrow w \cdot w$$

$$t_2 \leftarrow t_1 \cdot w$$

$$y \leftarrow t_2 + w + 5$$

たとえば $w = 2$ なら、

$$t_1 = 4, \quad t_2 = 8, \quad y = 15$$

制約として読む

$$t_1 - w^2 = 0$$

$$t_2 - t_1 w = 0$$

$$t_2 + w + 5 - y = 0$$

これらがすべて 0 なら、提出された w, t_1, t_2 は公開値 y と整合している。

左の 1 ステップが、右の 1 本の制約にそのまま対応する。

R1CS: 線形結合とは何か

R1CS では「線形結合」という言葉が何度も出る。これは変数を係数付きで足した式のこと。

定数項も同じ形で扱えるように、変数ベクトルの先頭に 1 を置いておく。

$$z = (1, y, w, t_1, t_2)$$

この z に対する線形結合は、成分ごとの係数を並べた式になる。

$$5 \cdot 1 - 1 \cdot y + 1 \cdot w + 0 \cdot t_1 + 1 \cdot t_2$$

何を書いたか

整理すると $t_2 + w + 5 - y$ 。前ページの 3 本目の制約 $t_2 + w + 5 - y = 0$ の左辺そのもの。係数の列 $(5, -1, 1, 0, 1)$ だけで、この式を指定できる。

R1CS: 掛け算 1 個の制約へ分解する

R1CS では、1 本の制約に掛け算を 1 個だけ許す。

$$(\text{線形結合}) \times (\text{線形結合}) = (\text{線形結合})$$

z は先頭に 1、続けて公開入力と witness を並べたベクトル。各制約の係数を行列 A, B, C の 1 行として並べると、制約全体が 1 本の式にまとまる。

$$(Az) \circ (Bz) = Cz$$

ここで $\circ$ は成分ごとの積 (Hadamard 積) で、 $(u \circ v)_i = u_i v_i$ 。

行と制約の対応

A の第 i 行が i 番目の制約の左因子、 B が右因子、 C が右辺の係数。式の第 i 成分が、そのまま i 番目の制約になる。

R1CS: 小例を行列として書く

$w^3 + w + 5 = y$ では $z = (1, y, w, t_1, t_2)^\top$ とおく。3本の制約を、3行の行列でまとめて書ける。

$$A = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 5 & -1 & 1 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}, \quad C = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

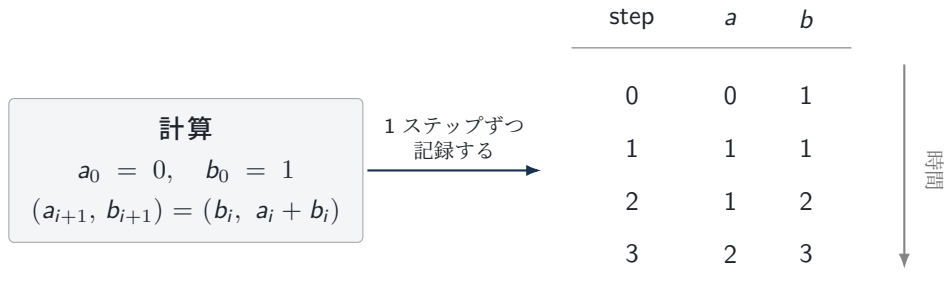
A の3行目 $(5, -1, 1, 0, 1)$ は、前ページで作った係数の列そのもの。 $(Az) \circ (Bz) = Cz$ を実際に掛けると、

$$\begin{pmatrix} w \\ t_1 \\ t_2 + w + 5 - y \end{pmatrix} \circ \begin{pmatrix} w \\ w \\ 1 \end{pmatrix} = \begin{pmatrix} t_1 \\ t_2 \\ 0 \end{pmatrix}$$

となり、 $w \cdot w = t_1$ 、 $t_1 \cdot w = t_2$ 、 $t_2 + w + 5 - y = 0$ の3本に戻る。3本目のように掛け算を含まない制約は、右因子を1にして(線形結合) $\times 1 = 0$ の形で載せる。

AIR: 計算を表として見る

AIR は、計算を「横に変数、縦に時間」の表として見る。この表を実行トレースと呼ぶ。R1CS が回路を見るのに対して、AIR は実行ログの全行が規則に従うかを見る。



- 1 行が 1 ステップの状態。行数が実行の長さ、列が変数の本数にあたる。
- prover がこの表を全部埋めて渡し、verifier は再計算せず、表が規則に従うかだけを確認する。

AIR: 表にかける2種類の制約

	step	a	b
境界制約 $a_0 = 0, b_0 = 1$	0	0	1
	1	1	1
	2	1	2
境界制約 最終行が公開出力と一致	3	2	3

遷移制約
$$a_{i+1} = b_i$$
$$b_{i+1} = a_i + b_i$$

この1組を、隣り合うすべての行の組に課す

row 1 $\rightarrow$ 2 なら $a_2 = b_1 = 1$ 、 $b_2 = a_1 + b_1 = 2$ で成り立っている。行が増えても書く式はこの1組のままで、制約の量は実行の長さによらない。

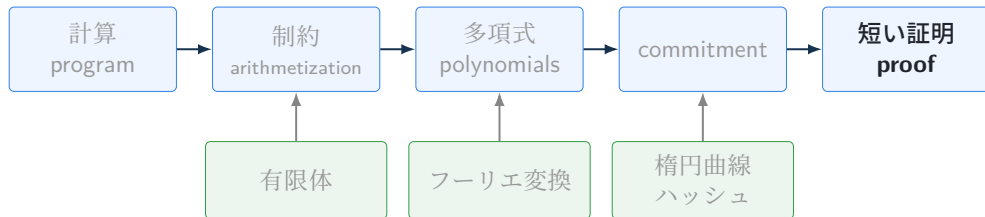
Arithmetization: 同じ計算を違う形に並べる

R1CS は並べ方の一つ。証明したい計算の性質に合わせて、別の並べ方も使われる。

方式	何として並べるか	見方	証明系
R1CS	掛け算 1 個の制約リスト	回路を細かい等式へ分解する	Groth16
AIR	時間方向の trace table	実行ログの各行が正しいか見る	STARK
Plonkish	wire 列、gate、selector の表	表計算のように行ごとのルールを見る	Plonk、Halo2
CCS	線形結合の積の和	上の 3 つを 1 つの記法へまとめる	Nova

Plonkish と CCS は appendix で扱う。どの方式も、計算の正しさを有限体上の制約として扱う。

ここからの話: 対話 ZK/FS



Schnorr: 問題設定

公開情報は、巡回群の生成元 g と $y = g^x$ 。prover は秘密の x を知っていることを示したい。

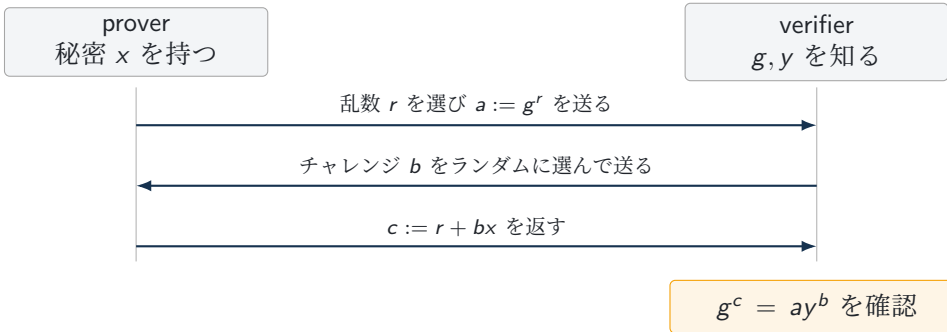
statement: $\exists x : g^x = y$, witness: x

目的

x を明かさずに、 y が g^x という形で作られていることを納得させる。

ここでは乗法記法で書く。楕円曲線では g^x を xP 、 g^r を rP と読めばよい。群の位数を q とし、指数 r, x, b は $\text{mod } q$ で計算する。

Schnorr: 対話式プロトコル



完全性

本当に $y = g^x$ なら、 $g^c = g^{r+bx} = g^r(g^x)^b = ay^b$ となり、検証式が成り立つ。

Schnorr: 例 小さな数で回す

$\mathbb{F}_{23}^*$ の中で、位数 $q = 11$ の元 $g = 2$ を使う。秘密 $x = 7$ 、公開値 $y = 2^7 = 128 \equiv 13 \pmod{23}$ 。

1 回の対話

- ① prover: 乱数 $r = 3$ を選び、 $a = 2^3 = 8$ を送る。
- ② verifier: チャレンジ $b = 5$ を送る。
- ③ prover: $c = r + bx = 3 + 5 \cdot 7 = 38 \equiv 5 \pmod{11}$ を返す。
- ④ verifier: 左辺は $g^c = 2^5 = 32 \equiv 9 \pmod{23}$ 。右辺は $13^2 \equiv 8$, $13^4 \equiv 8^2 \equiv 18$, $13^5 \equiv 18 \cdot 13 \equiv 4$ より $ay^b = 8 \cdot 4 = 32 \equiv 9$ 。両辺が一致するので受理。

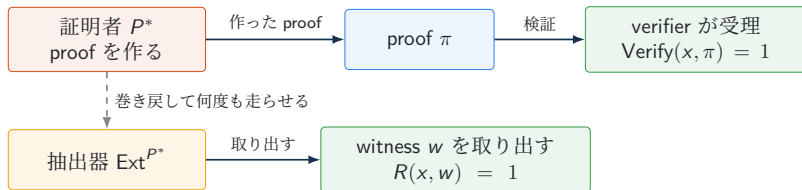
指数の計算は $\text{mod } q = 11$ 、群の計算は $\text{mod } p = 23$ で行っている点に注意。

知識健全性: 抽出器が witness を取り出せること

検証に通る応答を返せる prover は、本当に秘密を知っているのか。この「知っている」を、抽出器という道具で定義する。

知識健全性

verifier を十分な確率で受理させる証明者 P^* に対し、 P^* の振る舞いにアクセスできる抽出器 Ext^{P^*} が、 $R(x, w) = 1$ となる witness w を取り出せる。



取り出せたなら、witness は P^* の中にあったことになる。Schnorr なら、witness は指数 x 。

Schnorr: 抽出器を作る

Schnorr に対して、この抽出器 Ext を具体的に作る。使う道具は prover の巻き戻しである。

Ext^{P*} の手順

- ① P^* を走らせて a を受け取り、チャレンジ b を投げて応答 c を得る。
- ② a を送った直後まで P^* を巻き戻し、別のチャレンジ $b' \neq b$ を投げて応答 c' を得る。
- ③ 2つの応答から x を計算して出力する。

同じ a の下で c, c' がどちらも検証に通るなら、

$$c = r + bx, \quad c' = r + b'x \pmod{q}$$

の形をしている。差を取ると r が消えて $c - c' = (b - b')x$ となり、 $b \neq b'$ なので $b - b'$ の逆元を掛けて

$$x = (c - c')(b - b')^{-1} \pmod{q}$$

が手に入る。これが Ext の出力である。同じ a に対して2つのチャレンジへ答えられる prover は、実質的に x を知っている。

zero-knowledge: simulator が同じ見え方を作れること

proof から秘密が漏れていないことを、simulator という道具で定義する。

zero-knowledge

秘密 w を使う本物の実行記録と、公開情報 x だけから simulator S が作る記録が、verifier には見分けられない。

$$\text{Real}_{P,V^*}(x, w) \approx \text{Sim}_{S,V^*}(x)$$

この記録は公開情報 x だけから作れている。つまり verifier が見たものは、公開情報だけで説明がつく範囲に収まっている。

honest-verifier zero-knowledge

verifier がプロトコル通りにランダムなチャレンジを送る場合に限り、この見分けられなさを要求する版。

Schnorr で verifier が見るものは、会話 (a, b, c) がすべてである。この 3 つ組を x なしで作れば良い。

Schnorr: simulator を作る

Schnorr に対して、この simulator S を具体的に作る。順番を逆にして、答えから先に決めるのが鍵である。

S の手順

- ① チャレンジ b と応答 c をランダムに選ぶ。
- ② $a := g^c y^{-b}$ と置く。
- ③ 会話 (a, b, c) を出力する。

この a の作り方から、検証式

$$g^c = ay^b$$

は必ず成り立つ。本物の実行でも $c = r + bx$ より

$$a = g^r = g^{c-bx} = g^c (g^x)^{-b} = g^c y^{-b}$$

となり、同じ形をしている。

正直な verifier の下では b も c も一様にばらけるので、二つの記録は分布として完全に一致する。 x を一度も使わずに書けたこの simulator が、Schnorr の honest-verifier zero-knowledge を与える。

Fiat-Shamir 変換: チャレンジを hash で作る

verifier が選んでいたチャレンジ b を、ここまでの会話ログの hash で作る。

$$b := H(\text{ctx}, g, y, a)$$

ctx に入れるもの

プロトコル名、曲線や群の種類、証明したい statement などの文脈情報を入れる。

a は公開されるので、verifier も同じ b を手元で計算できる。prover は verifier の返事を待たずに、proof を 1 通で作れるようになる。

安全性

hash の出力を、verifier がその場で選んだ乱数と同じように扱う仮定が加わる。証明できる安全性は対話版より少し弱く、同じ目標ならパラメータを大きめに取る。

Schnorr: Fiat-Shamir 後の非対話版

Fiat-Shamir 後の prover:

- ① 乱数 r を選び、 $a := g^r$ を計算する。
- ② $b := H(\text{ctx}, g, y, a)$ を計算する。
- ③ $c := r + bx \pmod{q}$ を計算する。
- ④ proof として $\pi = (a, c)$ を送る。

verifier は a から同じ b を再計算し、

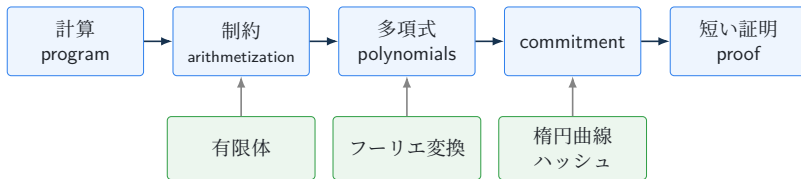
$$g^c = ay^b$$

を確認する。

これはそのまま署名になる

hash にメッセージ m も入れて $b := H(\text{ctx}, g, y, a, m)$ とすると、 $\pi = (a, c)$ は m への署名として使える。 x が秘密鍵、 $y = g^x$ が公開鍵にあたる。これが Schnorr 署名で、Bitcoin の BIP-340 や Ed25519 がこの系統にあたる。

全体像をもう一度



- 計算 → 制約: arithmetization が担う。代表例として R1CS を見た。
- 制約 → 多項式: 列を補間して 1 本の等式へ。係数と評価値の行き来をフーリエ変換が支える。
- 多項式 → commitment: KZG が多項式を群要素 1 個に固定する。土台は楕円曲線とペアリング。
- 対話 → 短い proof: Fiat-Shamir 変換でチャレンジを hash に置き換え、proof1 通にする。

一言で言うと

zkSNARK は、公開 statement に対して秘密 witness が存在することを、計算を制約と多項式へ翻訳し、commitment で短く固定して、短い proof として検証する仕組みである。

Further Reading

- [RareSkills ZK Book](#)
- [MoonMath Manual](#)
- [Ethereum EIP-4844: blob transactions](#)
- [Kate polynomial commitments \(Dankrad Feist\)](#)
- [クラウドを支えるこれからの暗号技術](#)
- 入門チュートリアル: [Circom](#), [Noir](#), [Halo2 book](#)



グループワーク: 4×4 数独 ZKP

2	1		
		1	2
		4	3
4	3		

4×4 数独の例

課題

4×4 数独の解を知っていることの ZKP を作ってみよう。まずはトランプカードを使った物理世界での ZKP プロトコルを考えてみる。時間があれば数独の制約を R1CS で記述する。



プロトコルが決まったら、次の三つを言葉で説明してみる。

完全性

正しい解なら必ず通るか

健全性

解を持たない人が通る確率は

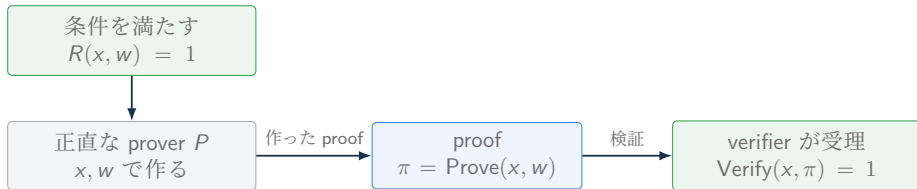
ゼロ知識性

解なしの simulator を作れるか

zkSNARK 定式化: Completeness

Completeness は、正しい statement と witness から正直に作った proof を、verifier が必ず受理するという性質。

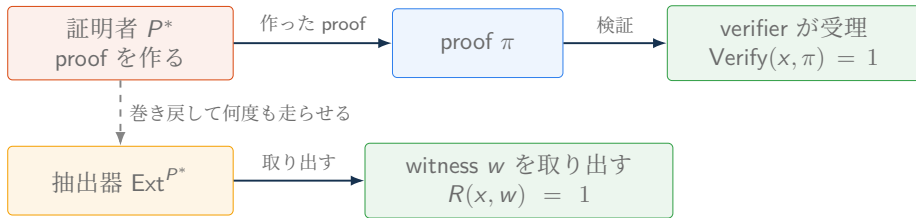
$$R(x, w) = 1 \implies \text{Verify}(x, \text{Prove}(x, w)) = 1$$



x は公開された statement、 w は statement を正しくする witness。条件を本当に満たしている prover が、プロトコルの都合で落とされたままにならないことを保証する。

zkSNARK 定式化: Knowledge soundness

Knowledge soundness は、有効な proof を作れるなら対応する witness を知っているはずだ、という性質。



証明者 P^* が verifier を十分な確率で受理させられるなら、 P^* の振る舞いにアクセスする抽出器 Ext^{P^*} が $R(x, w) = 1$ となる witness w を取り出せる。

zkSNARK 定式化: Zero-knowledge

zero-knowledge は、「本物の実行」と「シミュレーションで作った見かけの実行」を見分けられないという性質である。

$$\text{Real}_{P,V^*}(x, w) \approx \text{Sim}_{S,V^*}(x)$$

$\text{Real}_{P,V^*}(x, w)$ は、prover が witness w を使って作る本物の実行記録を表す。 $\text{Sim}_{S,V^*}(x)$ は、simulator が公開情報 x とシミュレーション用の設定情報から作る見かけの実行記録を表す。zero-knowledge では、verifier はこの二つの記録を見分けられない。

見分けられなさの強さは三段階ある。

- Perfect: 二つの確率分布が完全に一致する。
- Statistical: 確率分布の差が十分に小さい。
- Computational: 効率的に区別するアルゴリズムが存在しない。

有限体: 拡張 Euclid 互除法で逆元を求める

素数 p と $0 < x < p$ は互いに素なので、次を満たす整数 a, b が見つかる。

$$ax + bp = 1$$

$\text{mod } p$ では $bp \equiv 0$ なので $ax \equiv 1 \pmod{p}$ 。つまり a が x の逆元になる。

例: $\mathbb{F}_{13}$ で 5^{-1} を求める

余りを取りながら 1 まで下りる。

$$13 = 2 \cdot 5 + 3, \quad 5 = 1 \cdot 3 + 2, \quad 3 = 1 \cdot 2 + 1$$

下の式から順に、1 を 13 と 5 の組み合わせへ書き直す。

$$1 = 3 - 1 \cdot 2$$

$$= 3 - 1 \cdot (5 - 1 \cdot 3) = 2 \cdot 3 - 1 \cdot 5$$

$$= 2 \cdot (13 - 2 \cdot 5) - 1 \cdot 5 = 2 \cdot 13 - 5 \cdot 5$$

$\text{mod } 13$ で見ると $-5 \cdot 5 \equiv 1$ 。よって $5^{-1} = -5 \equiv 8$ 。検算: $5 \cdot 8 = 40 \equiv 1 \pmod{13}$ 。

余りを取る回数は $\log p$ に比例する。256bit の素数でも数百回の割り算で逆元が求まる。

Lagrange 補間: 基底と復元式

異なる点 $x_0, \dots, x_d$ と値 $y_0, \dots, y_d$ があるとする。まず、Lagrange 基底 L_i は評価点で次の振る舞いをするように作る。

$$L_i(x_j) = \begin{cases} 1 & (i = j), \\ 0 & (i \neq j). \end{cases}$$

この性質があれば、 $f(x_i) = y_i$ を満たす多項式を

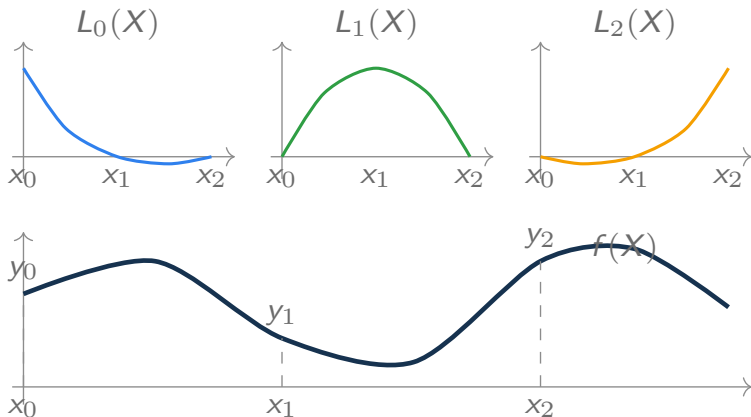
$$f(X) = \sum_{i=0}^d y_i L_i(X)$$

と書ける。
ちなみに、各 $L_i(X)$ は具体的には

$$L_i(X) = \prod_{\substack{0 \leq j \leq d \\ j \neq i}} \frac{X - x_j}{x_i - x_j}$$

と書ける。有限体では $x_i - x_j \neq 0$ なら逆元で割れる。

Lagrange 補間: 基底を重ねて $f(X)$ を作る



$$f(X) = y_0 L_0(X) + y_1 L_1(X) + y_2 L_2(X)$$

Lagrange 補間: 3 点から 2 次式を復元する

3 点 $(0, 1)$, $(1, 2)$, $(2, 5)$ を通る 2 次以下の多項式を、Lagrange 補間で作る。まず各基底を計算する。分母は「自分の点と他の点との差」の積。

$$L_0(X) = \frac{(X-1)(X-2)}{(0-1)(0-2)} = \frac{(X-1)(X-2)}{2}, \quad L_1(X) = \frac{X(X-2)}{(1-0)(1-2)} = -X(X-2)$$

$$L_2(X) = \frac{X(X-1)}{(2-0)(2-1)} = \frac{X(X-1)}{2}$$

値 y_i を重みにして足し合わせる。

$$f(X) = 1 \cdot L_0(X) + 2 \cdot L_1(X) + 5 \cdot L_2(X) = X^2 + 1$$

検算: $f(0) = 1$, $f(1) = 2$, $f(2) = 5$ 。同じ手順が次数 d 以下・ $d+1$ 点でそのまま使え、有限体では割り算を逆元で行う。

フーリエ変換: 逆変換で使う補題

n は $\mathbb{F}_p$ で逆元を持つとする。位数 n の 1 の根 ω について、冪の和は次の形になる。

1 の根の冪の和

$$S_k = \sum_{j=0}^{n-1} \omega^{kj} = \begin{cases} n & (k \equiv 0 \pmod{n}), \\ 0 & (k \not\equiv 0 \pmod{n}). \end{cases}$$

$k \equiv 0$ なら各項が 1 なので $S_k = n$ 。 $k \not\equiv 0$ のときは、両辺に $1 - \omega^k$ を掛けると

$$(1 - \omega^k)S_k = \sum_{j=0}^{n-1} \omega^{kj} - \sum_{j=0}^{n-1} \omega^{k(j+1)} = 1 - \omega^{kn} = 0.$$

$\omega^k \neq 1$ なので $1 - \omega^k$ は 0 以外。したがって $S_k = 0$ 。

フーリエ変換: 逆変換が元の係数を返す

補題を使うと、逆変換の式が元の係数へ戻ることを確かめられる。 $y_j = f(\omega^j) = \sum_{\ell=0}^{n-1} a_\ell \omega^{\ell j}$ とすると、

$$\begin{aligned}\frac{1}{n} \sum_{j=0}^{n-1} y_j \omega^{-ij} &= \frac{1}{n} \sum_{j=0}^{n-1} \left(\sum_{\ell=0}^{n-1} a_\ell \omega^{\ell j} \right) \omega^{-ij} \\ &= \frac{1}{n} \sum_{\ell=0}^{n-1} a_\ell \sum_{j=0}^{n-1} \omega^{(\ell-i)j} \quad (\text{内側の和が補題の } S_{\ell-i}) \\ &= \frac{1}{n} \left(a_i n + \sum_{\ell \neq i} a_\ell \cdot 0 \right) = a_i.\end{aligned}$$

$\ell = i$ の項だけが n を返し、他の項は 0 で消える。

フーリエ変換: 例 $\mathbb{F}_7$ で変換して戻す

$\mathbb{F}_7$ では $\omega = 2$ が位数 3 の 1 の根になる。多項式 $f(X) = 3 + X + 2X^2$ を $H = \{1, 2, 4\}$ 上で評価すると、

$$y_0 = f(1) = 6, \quad y_1 = f(2) = 6, \quad y_2 = f(4) = 4 \pmod{7}$$

逆変換 $a_i = \frac{1}{n} \sum_{j=0}^{n-1} y_j \omega^{-ij}$ を定義どおりに計算する。 $n = 3$ の逆元は $3^{-1} = 5$ 、 ω の逆元は $2^{-1} = 4$ で、その冪は $4^0 = 1$, $4^1 = 4$, $4^2 \equiv 2$, $4^3 \equiv 1$, $4^4 \equiv 4$ 。

$$a_0 = 5(6 \cdot 1 + 6 \cdot 1 + 4 \cdot 1) = 5 \cdot 16 \equiv 5 \cdot 2 = 10 \equiv 3$$

$$a_1 = 5(6 \cdot 1 + 6 \cdot 4 + 4 \cdot 2) = 5 \cdot 38 \equiv 5 \cdot 3 = 15 \equiv 1$$

$$a_2 = 5(6 \cdot 1 + 6 \cdot 2 + 4 \cdot 4) = 5 \cdot 34 \equiv 5 \cdot 6 = 30 \equiv 2$$

やったこと

係数列 $(3, 1, 2)$ を評価値列 $(6, 6, 4)$ へ変換し、逆変換で $(3, 1, 2)$ が戻ってきた。

FFT: フーリエ変換を速く計算する

フーリエ変換の式

$$y_j = \sum_{i=0}^{n-1} a_i \omega^{ij} \quad (j = 0, \dots, n-1)$$

をそのまま計算すると、1点あたり n 回の掛け算を n 点分繰り返すので、掛け算はおよそ n^2 回かかる。

方法	掛け算の回数	$n = 2^{20}$ のとき
定義どおりの計算	n^2	約 10^{12} 回
FFT	$\frac{n}{2} \log_2 n$	約 10^7 回

証明生成では n が 2^{20} を超える変換を何度でも行うため、 n^2 では時間が足りなくなる。FFT は、評価点集合 $H = \{1, \omega, \dots, \omega^{n-1}\}$ の規則性を利用して、同じ y を高速に求めるアルゴリズム。

FFT: 評価点は $\pm$ の対をなす (観察 1)

n を偶数とする。 $\omega^{n/2}$ は 2 乗すると $\omega^n = 1$ になり、位数 n の仮定から 1 とは異なるので

$$\omega^{n/2} = -1, \quad \omega^{j+n/2} = -\omega^j.$$

つまり評価点集合 H は $n/2$ 組の対 $\pm\omega^j$ ($j = 0, \dots, n/2 - 1$) に分かれる。

さらに、対になった 2 点は 2 乗すると同じ点に移る。

$$(\pm\omega^j)^2 = \omega^{2j}$$

ω^2 は位数 $n/2$ の 1 の根なので、2 乗後の点集合は半分のサイズの評価点集合 $H' = \{1, \omega^2, \omega^4, \dots\}$ になる。

例: $\mathbb{F}_5$ で $\omega = 2$ (位数 4)

$H = \{1, 2, 4, 3\}$ 。 $4 = -1$ 、 $3 = -2$ なので対は $(1, 4)$ と $(2, 3)$ 。 2 乗すると $\{1, 4\}$ の 2 点に半減する。

FFT: 多項式は偶数次と奇数次に分けられる (観察 2)

係数を偶数次と奇数次に振り分けると、どんな f も次の形に書ける。

$$f(X) = f_{\text{even}}(X^2) + X f_{\text{odd}}(X^2)$$

例: $f(X) = 1 + 2X + 3X^2 + 4X^3$ なら

$$f_{\text{even}}(Y) = 1 + 3Y, \quad f_{\text{odd}}(Y) = 2 + 4Y, \quad f(X) = (1 + 3X^2) + X(2 + 4X^2).$$

この形に対する 2 点 $X = \pm\omega^j$ を代入すると、 $X^2 = \omega^{2j}$ が共通なので

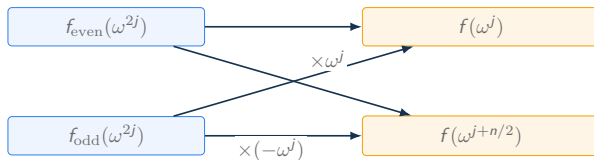
$$f(\omega^j) = f_{\text{even}}(\omega^{2j}) + \omega^j f_{\text{odd}}(\omega^{2j}), \quad f(-\omega^j) = f_{\text{even}}(\omega^{2j}) - \omega^j f_{\text{odd}}(\omega^{2j}).$$

半分サイズの多項式 $f_{\text{even}}, f_{\text{odd}}$ の値を 1 回ずつ求めれば、2 点分の値が同時に得られる。

FFT: 蝶形で2つの観察を組み合わせる

$E_j = f_{\text{even}}(\omega^{2j})$ 、 $O_j = f_{\text{odd}}(\omega^{2j})$ と置くと、 $j = 0, \dots, n/2 - 1$ について

$$y_j = E_j + \omega^j O_j, \quad y_{j+n/2} = E_j - \omega^j O_j.$$



この組み合わせ方を蝶形 (butterfly) と呼ぶ。掛け算は $\omega^j O_j$ の1回だけ。残る仕事は E_j, O_j を求めること、つまり $f_{\text{even}}, f_{\text{odd}}$ を H' 上で評価することで、これは半分のサイズ $n/2$ の同じ問題が2つ。同じ手順を再帰的に繰り返せばよい。

FFT: 再帰の全体像と計算量

サイズ n の変換は、蝶形 $n/2$ 個とサイズ $n/2$ の変換 2 つに置き換わる。サイズ 1 まで割り切ったら、定数多項式の値は係数そのものなので計算は終わる。

段	部分問題	蝶形の掛け算
1 段目	サイズ n が 1 つ	$n/2$ 回
2 段目	サイズ $n/2$ が 2 つ	$n/2$ 回
$\vdots$	$\vdots$	$\vdots$
$\log_2 n$ 段目	サイズ 2 が $n/2$ 個	$n/2$ 回

どの段でも蝶形の掛け算は合計 $n/2$ 回で、段数は $\log_2 n$ 。掛け算の総数は

$$\frac{n}{2} \log_2 n.$$

半分に割り続けるため、radix-2 FFT では $n = 2^m$ となる評価点集合を選ぶ。

FFT: 例 (1/2) $\mathbb{F}_5$ 、 $n = 4$ で準備する

半分に割り続けるにはサイズが 2^m の例が要る。 $\mathbb{F}_5$ では $\omega = 2$ が位数 4 の 1 の根になる。

$$2^0 = 1, \quad 2^1 = 2, \quad 2^2 = 4, \quad 2^3 = 3, \quad 2^4 = 1$$

係数列 $a = (1, 2, 3, 4)$ 、つまり

$$f(X) = 1 + 2X + 3X^2 + 4X^3$$

を $H = \{1, 2, 4, 3\}$ 上で評価する。

Step 1: 偶数次と奇数次に分ける

$$f_{\text{even}}(Y) = 1 + 3Y, \quad f_{\text{odd}}(Y) = 2 + 4Y$$

この 2 つを、半分の評価点集合 $H' = \{1, \omega^2\} = \{1, 4\}$ 上で評価すればよい。

FFT: 例 (2/2) 蝶形で組み上げる

Step 2: サイズ 2 の評価 ($\omega^2 = 4 = -1$ を代入するだけ)

$$\begin{aligned} E_0 &= f_{\text{even}}(1) = 1 + 3 = 4, & E_1 &= f_{\text{even}}(4) = 1 + 3 \cdot 4 = 13 = 3 \\ O_0 &= f_{\text{odd}}(1) = 2 + 4 = 6 = 1, & O_1 &= f_{\text{odd}}(4) = 2 + 4 \cdot 4 = 18 = 3 \end{aligned}$$

Step 3: 蝶形でサイズ 4 に合成 ($\omega = 2$)

$$\begin{aligned} y_0 &= E_0 + \omega^0 O_0 = 4 + 1 = 5 = 0, & y_2 &= E_0 - \omega^0 O_0 = 4 - 1 = 3 \\ y_1 &= E_1 + \omega^1 O_1 = 3 + 2 \cdot 3 = 9 = 4, & y_3 &= E_1 - \omega^1 O_1 = 3 - 2 \cdot 3 = -3 = 2 \end{aligned}$$

結果は $y = (0, 4, 3, 2)$ 。検算として定義どおりに $f(2) = 1 + 4 + 12 + 32 = 49 = 4$ を計算すると y_1 と一致する。

FFT: IFFT と押さえどころ

逆変換

$$a_i = \frac{1}{n} \sum_{j=0}^{n-1} y_j \omega^{-ij}$$

は、 ω を ω^{-1} に置き換えた同じ形の和である。 ω^{-1} も位数 n の 1 の根なので、同じ FFT を実行して最後に各成分へ $1/n$ を掛ければよい。これを IFFT と呼ぶ。

最低限覚えること

FFT/IFFT は、係数列と評価値列の行き来を $\frac{n}{2} \log_2 n$ 回の掛け算で行う。鍵は「評価点が $\pm$ の対をなす」ことと「偶数次・奇数次への分割」で、サイズ n の変換が半分のサイズの変換 2 つに落ちる。

Commitment: 定式化 (binding • hiding)

封筒の直感を、3つのアルゴリズムと2つの性質の式に写す。公開パラメータを使う構成では、次のように書ける。

$$pp \leftarrow \text{Setup}(1^\lambda), \quad (c, d) \leftarrow \text{Commit}(pp, m), \quad b \leftarrow \text{Verify}(pp, c, m, d)$$

記号	意味
pp	公開パラメータ
c	commitment
d	decommitment / opening information。乱数、path、proof などの確認用補助情報
b	検証結果。受理なら 1、拒否なら 0

Binding

効率的な攻撃者が、同じ commitment を二つの異なる message として open することは困難である。このような組を見つける確率は negligible である。

$$\text{Verify}(pp, c, m, d) = 1,$$

$$\text{Verify}(pp, c, m', d') = 1, \quad m \neq m'$$

Hiding

c だけを見ても、どちらの message から作られたかを当てにくい。

$$\left| \Pr[b' = b] - \frac{1}{2} \right| \text{ が小さい}$$

Commitment: Merkle tree

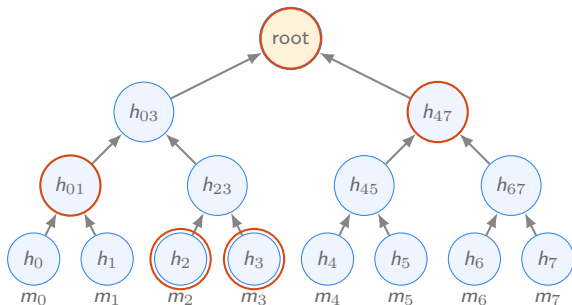
大きな配列 $(m_0, m_1, \dots, m_{n-1})$ 全体を固定したいとき、配列全体を毎回 open すると検証の負担が大きい。Merkle tree は、葉から親へ hash を重ねて、配列全体を 1 つの root にまとめる。

root を commitment として公開する。

$$h_i = \text{Hash}(m_i), \quad \text{root} = \text{Hash}(\dots)$$

1 要素だけを open するときは、root まで再計算するために必要な隣接 hash だけを渡す。

Merkle tree では、配列の一部だけを $O(\log n)$ の proof で open できる。



Commitment: 例 Merkle proof の検証

8 個の leaf の木で、 $root$ と index 2 が公開されているとする。prover が渡すのは、開く値 m_2 と、道中の隣接 hash 3 個 (path)。

$$m_2, \quad (h_3, \text{right}), \quad (h_{01}, \text{left}), \quad (h_{47}, \text{right})$$

verifier の再計算: 葉から root へ上る

$$h_2 = \text{Hash}(m_2), \quad h_{23} = \text{Hash}(h_2 \parallel h_3), \quad h_{03} = \text{Hash}(h_{01} \parallel h_{23}), \quad root' = \text{Hash}(h_{03} \parallel h_{47})$$

最後に $root' = root$ を確認する。一致すれば、 m_2 が index 2 に固定されていたと納得できる。

渡す hash は木の高さぶんの $\log_2 8 = 3$ 個。leaf が 100 万個でも 20 個で済む。

実装上の注意

左右の順序、leaf のエンコード、domain separation は、いずれも一意に決めておく。

Commitment: Pedersen

楕円曲線上の2つの基点 G, H を使い、値 m と乱数 r をスカラー倍で混ぜる。

$$\text{Com}(m; r) = mG + rH$$

Hiding

乱数 r によって、同じ m でも異なる点になる。

Binding

G と H の離散対数関係を誰も知らないことが前提になる。

加法準同型性: 足し算がそのまま通る

$$\text{Com}(a; r_a) + \text{Com}(b; r_b) = (a + b)G + (r_a + r_b)H = \text{Com}(a + b; r_a + r_b)$$

commitment を開かずに、中身の和の commitment を作れる。

Commitment: Pedersen の binding は離散対数に立つ

同じ commitment を二通りに開けたと仮定する。

$$mG + rH = m'G + r'H, \quad (m, r) \neq (m', r')$$

移項すると

$$(m - m')G = (r' - r)H$$

となり、 $H = \alpha G$ と書いたときの $\alpha = (m - m')(r' - r)^{-1}$ が計算できてしまう。つまり二重 open は、 H の離散対数を解いたのと同じこと。離散対数問題が難しく、誰も α を知らない限り、binding は破れない。

もう一つの注意

値は群の位数 q を法として扱われる。整数としての範囲を保証したい場合は range proof や追加制約が必要になる。

Commitment: KZG の hiding 版

基本版の $C = f(\tau)P$ は f だけで決まるので、同じ f なら誰が作っても同じ C になる。値を隠したいときは、独立な基点 H と乱数多項式 $r(X)$ を混ぜる。

$$C = f(\tau)P + r(\tau)H$$

SRS には $H, \tau H, \dots$ 側の powers も必要になる。

$f(z) = y, r(z) = \rho$ を開くときは、商を 2 本作って束ねる。

$$q_f(X) = \frac{f(X) - y}{X - z}, \quad q_r(X) = \frac{r(X) - \rho}{X - z}, \quad \pi = q_f(\tau)P + q_r(\tau)H$$

検証式は基本版と同じ形になる。

$$e(C - yP - \rho H, Q) = e(\pi, \tau Q - zQ)$$

Pedersen で乱数 r を混ぜたのと同じ発想である。

Commitment: KZG の batch 版

複数の点 $z_1, \dots, z_m$ での値 $f(z_i) = y_i$ を、proof1 個でまとめて示せる。補間多項式 $Y(X)$ と積多項式 $D(X)$ を置く。

$$Y(z_i) = y_i, \quad D(X) = \prod_{i=1}^m (X - z_i)$$

すべての z_i で $f(z_i) = y_i$ が成り立つことは、 $f(X) - Y(X)$ が $D(X)$ で割り切れることと言い換えられる。

$$q(X) = \frac{f(X) - Y(X)}{D(X)}, \quad \pi = q(\tau)P$$

検証は 1 つの pairing 等式で行う。

$$e(C - Y(\tau)P, Q) = e(\pi, D(\tau)Q)$$

この式をそのまま使うには、 $D(\tau)Q$ を作るための G_2 側 powers も必要になる。

Commitment: 4 方式の比較

Scheme	対象	主な仮定	証明サイズ	強み	注意点
Hash	1 個の message	第二原像困難性、衝突困難性	open 対象に依存	単純	小さい message は乱数が必要になる
Merkle	配列・木	第二原像困難性、衝突困難性	$O(\log n)$	部分 open が得意	順序、salt、domain separation
Pedersen	値・ベクトル	離散対数問題、基点間の未知関係	開く値に依存	hiding、加法準同型	乱数再利用、範囲制約
KZG	多項式	ペアリング、SDH 系仮定、SRS	π は $O(1)$	短い proof、高速検証	opening data と検証計算は点数に依存

最低限覚えること

Merkle はデータ構造、Pedersen は準同型 commitment、KZG は多項式 commitment として覚える。

Plonkish: 表として制約を書く

Plonkish arithmetization では、計算を表として配置する。

row	a	b	c	q_{mul}	q_{add}
0	a_0	b_0	c_0	1	0
1	a_1	b_1	c_1	0	1
2	a_2	b_2	c_2	1	0

- row は計算ステップを置く行である。
- a, b, c は wire 値を置く advice column であり、prover が witness や中間値から埋める。
- $q_{\text{mul}}, q_{\text{add}}$ は fixed column に置く selector であり、その行でどの gate を有効にするかを選ぶ。gate は各行で満たすべき式である。
- 公開入力 instance column に置く。別々のセルが同じ値であることは copy constraint で保証する。

Plonkish: gate を式で見る

各行で満たすべき gate は、selector を係数にした 1 本の式で書ける。

$$q_M ab + q_L a + q_R b + q_O c + q_C = 0$$

gate	q_M	q_L	q_R	q_O	q_C	得られる制約
mul	1	0	0	-1	0	$ab - c = 0$
add	0	1	1	-1	0	$a + b - c = 0$
const	0	1	0	0	$-k$	$a - k = 0$

selector の値を変えるだけで、同じ 1 本の式が行ごとに違う制約になる。たとえば mul 行では $q_M = 1$, $q_O = -1$ だけが立ち、 $ab - c = 0$ が残る。

Plonkish: 列を多項式として扱う

行数を n 、評価領域を $H = \{1, \omega, \dots, \omega^{n-1}\}$ とする。列 a を、各行の値を評価値として持つ多項式 $A(X)$ で表す。

$$A(\omega^i) = a_i$$

値の列から多項式を作るこの補間が IFFT である。

$B, C, Q_M, \dots$ も同様に多項式になり、gate の式は 1 本の多項式にまとまる。

$$G(X) = Q_M AB + Q_L A + Q_R B + Q_O C + Q_C$$

すべての行で gate が成り立つことは、

$$G(\omega^i) = 0 \quad (i = 0, \dots, n-1)$$

つまり「 G が H 上で 0」と言い換えられる。多数の制約が多項式の等式に翻訳され、ランダム点での検査や commitment の道具が使えるようになる。

Plonkish: 小例を配置する

$w^3 + w + 5 = y$ を 3 本の advice 列 a, b, c に置く。gate は 2 入力なので、足し算 $y = t_2 + w + 5$ は $t_3 = w + 5$ と $y = t_2 + t_3$ の 2 行に分ける。

row	a	b	c	gate
0	w	w	t_1	$a \cdot b = c$
1	t_1	w	t_2	$a \cdot b = c$
2	w	5	t_3	$a + b = c$
3	t_2	t_3	y	$a + b = c$

行ごとの gate はこれで揃った。ただし、row 0 の c と row 1 の a のように、行をまたいで「同じ値」を使っている箇所が残っている。

Plonkish: copy constraint でセルどうしを結ぶ

表のセルは、場所が違えば別の変数。row 0 で計算した t_1 を row 1 で使うには、「row 0 の c と row 1 の a は同じ値」だと制約で言う必要がある。これが copy constraint。

小例で結ぶセル

t_1 : row 0 の c と row 1 の a

w : row 0 の a, b , row 1 の b , row 2 の a

t_2 : row 1 の c と row 3 の a

t_3 : row 2 の c と row 3 の b

y : row 3 の c と公開入力 y

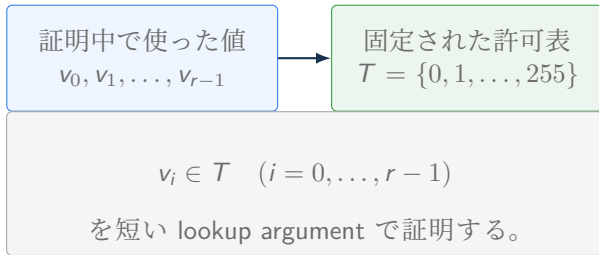
また、row 2 の $b = 5$ は fixed column や constant gate で固定する。

注意

表に同じ記号を書くのは人間向けの表記。証明の中で効かせるには、equality も制約として入れる。

Lookup: 値が許可表に入ること示す

lookup は、証明中で使った値が、あらかじめ固定した表に含まれることを示す制約である。複雑な計算を毎回 gate で展開する代わりに、「この行は許可表のどれかと一致する」と確認する。



- 表 T は fixed column として回路側に置く。
- 問い合わせる値 v_i は advice column に置く。
- 実装では並べ替えや多重集合の一致を使うが、意味は table membership である。

値の組も同じ考え方で扱える。例えば $(\text{opcode}, a, b, c) \in T_{\text{op}}$ と書けば、1 行の命令が許可された命令表に載っていることを示す。

Lookup: 応用例

小さな表で書ける関係を回路の中で何度も使う場面では、lookup が特に安く済む。

用途	表に置くもの	何を簡潔に書けるか
range check	$T_{\text{byte}} = \{0, 1, \dots, 255\}$	$v \in T_{\text{byte}}$ と書くことで、byte 値であることを示す。bit ごとの制約を毎回並べない。
ハッシュ関数の部品	$T_{\text{sbox}} = \{(x, S(x))\}$	S-box や小さな非線形変換を、入力と出力の対応表として確認する。
zkVM の命令表	$T_{\text{op}} = \{(\text{op}, a, b, c)\}$	$(\text{op}, a, b, c) \in T_{\text{op}}$ として、opcode ごとの意味を表にまとめる。

回路の正しさは 3 種類の制約で分担する。gate は行ごとの等式を、copy constraint はセル同士の同一性を、lookup は値が許可表に入っていることを受け持つ。

CCS: 制約形式を一般化する

CCS(Customizable Constraint Systems) は、R1CS・AIR・Plonkish を 1 つの記法へまとめる一般化である。

$$\sum_{i=1}^q c_i \cdot \bigcirc_{j \in S_i} (M_j z) = 0$$

読み方: 行列 M_j が線形結合の束を作り、 S_i で選んだものどうしを掛け、係数 c_i を付けて足す。 S_i は同じ添字を複数回使える多重集合として扱う。

何が嬉しいか

R1CS、Plonkish 風の gate、高 degree gate を、「線形結合の積の和」という同じ記法で扱える。folding の土台になる。

CCS: データ

有限体を $\mathbb{F}$ とする。CCS では次のデータを固定する。

- 行数 m と変数数 n 。
- 行列 $M_1, \dots, M_t \in \mathbb{F}^{m \times n}$ 。
- 項を指定する添字集合 $S_1, \dots, S_q$ 。厳密には多重集合として扱う。
- 係数 $c_1, \dots, c_q \in \mathbb{F}$ 。

公開入力 x 、witness w 、定数 1 を含めて

$$z = (w, x, 1) \in \mathbb{F}^n$$

と置く。順番は方式や文献ごとの約束で、必要な値がすべて z に入っていればよい。

CCS: 式を成分ごとに読む

CCS の制約は

$$\sum_{i=1}^q c_i \cdot \bigcirc_{j \in S_i} (M_j z) = 0 \in \mathbb{F}^m$$

である。

Hadamard 積は成分ごとの積:

$$(u \circ v)_r = u_r v_r$$

成分ごとに書くと、各行 r について

$$\sum_{i=1}^q c_i \prod_{j \in S_i} (M_j z)_r = 0$$

を要求している。

CCS: R1CS は特別な場合

R1CS は

$$(Az) \circ (Bz) - Cz = 0$$

と書ける。

CCS で

$$M_1 = A, \quad M_2 = B, \quad M_3 = C$$

$$S_1 = \{1, 2\}, \quad S_2 = \{3\}, \quad c_1 = 1, \quad c_2 = -1$$

と置くと、

$$(M_1 z) \circ (M_2 z) - (M_3 z) = 0$$

となり、R1CS を含む。

CCS: Plonkish gate も同じ形で書ける

Plonkish の gate

$$q_{Mab} + q_L a + q_R b + q_O c + q_C = 0$$

も、行ごとに見れば線形結合の積の和。

$$(M_1 z) \circ (M_2 z) + (M_3 z) + (M_4 z) + (M_5 z) + (M_6 z) = 0$$

高 degree gate も同じ見方で扱える。例えば

$$a_i b_i c_i - d_i = 0$$

は

$$(M_1 z) \circ (M_2 z) \circ (M_3 z) - (M_4 z) = 0$$

と書ける。

CCS: folding で役立つ理由

folding scheme では、二つの証明対象を一つの証明対象へまとめる。そのとき、制約が共通の形で書かれていると扱いやすい。

$$\sum_i c_i \cdot \prod_{j \in S_i} \text{linear form}_{i,j}(z) = 0$$

分解できる操作

- 線形結合を作る。
- 同じ行どうしで掛ける。
- 係数を掛けて足す。

R1CS、Plonkish 風の gate、高 degree gate を共通インターフェースへ持ち込みやすい。

Arithmetization: 4 方式の比較

方式	主な表現	得意な計算	使われる証明系	理解
R1CS	乗算制約の集合	算術回路	Groth16	回路を制約に分解する
AIR	実行トレースと遷移	VM、繰り返し計算	STARK	計算履歴が正しいか見る
Plonkish	gate と copy constraint	汎用回路、lookup	Plonk、Halo2	表計算のように制約を書く
CCS	一般化された制約	folding-friendly な表現	Nova	複数方式をまとめる抽象化

最低限覚えること

arithmetization は、計算を有限体上の制約へ翻訳する工程である。方式ごとに「何を自然に表せるか」が違う。

記法と用語のまとめ

記法	意味
x	公開入力。verifier にも見える値。
w	witness。statement を成り立たせる補助情報。
π	proof。verifier へ渡す短い証拠。
$R(x, w) = 1$	関係 R が、公開入力 x と witness w を受理すること。 statement は通常 $\exists w : R(x, w) = 1$ 。
$\mathbb{F}_p$	素数 p 個の元からなる有限体。
$\mathbb{F}_p^*$	$\mathbb{F}_p$ から 0 を除いた集合。掛け算について群になる集合。
$\langle g \rangle$	元 g が生成する巡回群。
g^a, aP	乗法群と楕円曲線の加法群で、秘密スカラーを掛けた公開値。
$E/k, e$	体 k 上の楕円曲線と、 $e : G_1 \times G_2 \rightarrow G_T$ のようなペアリング。

用語	意味
statement	検証者が確認したい公開された主張。
proof / argument	statement が正しいこと、または witness を知っていることを示す証拠。witness の秘匿は zero-knowledge 性が担う。
transcript hash	protocol 名、statement、commitment などの公開ログから作る hash。
commitment	値や多項式を短い値に固定し、必要な部分だけ open する仕組み。
binding / hiding	commit 後に別の値へ開けない性質と、commitment から値が漏れない性質。
arithmetization	計算を有限体上の制約へ翻訳する工程。
R1CS / AIR / Plonkish / CCS	計算を制約として表すための代表的な表現形式。
離散対数問題	公開値から秘密スカラーを求める問題。
計算	Diffie-Hellman 型の値を計算する問題。
Diffie-Hellman 問題	
判定	Diffie-Hellman 型の値かどうかを判定する問題。
Diffie-Hellman 問題	
SRS	KZG などを使う Structured Reference String。公開パラメータ。